

Backend & Model Integration Roadmap

Production-grade AI platform · Multi-model intelligence routing · Phase-by-phase implementation
Version 1.0 · 2026

THE CORE IDEA

CAT AI is not a simple multi-model wrapper. It is an **intelligence router** — each AI provider is assigned to the backend modules where it genuinely excels. GPT-4o owns conversational NLP and tool calling. Gemini handles multimodal and long-context workloads. Claude drives narrative reasoning and code generation. Groq powers all asynchronous background tasks at 80–90% lower cost. A unified **BaseProvider** abstract class normalizes all provider APIs so the orchestration layer is completely provider-agnostic — swapping or adding a model requires changing one adapter, not the orchestration code.

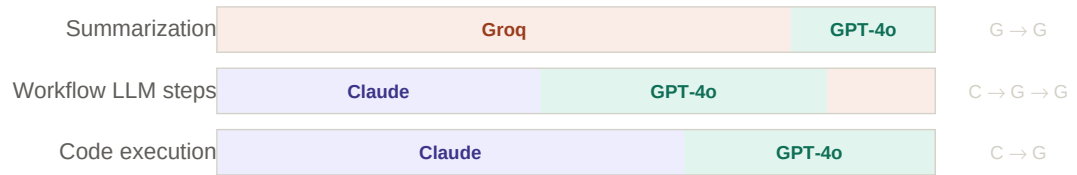
MODEL INTELLIGENCE ROUTING

GPT-4o OpenAI · General NLP [NLP ↗] [Tool calling] [Function calls] [Structured output] Primary model for user-facing chat, tool execution, and embedding.	Gemini 1.5 Google · Vision + Long ctx [Vision ↗] [1M tokens] [Multimodal] [Image-heavy docs] Routes image-heavy PDFs and long-context conversations > 70% window.
Claude 3.5+ Anthropic · Reasoning [Narrative ↗] [Long tasks] [Code] [Reasoning] Narrative tasks, code sandbox, approval summaries, circuit-breaker fallback.	Groq Llama3 Groq · Speed + Cost [Speed ↗] [Cheap] [Summarize] [Background jobs] All async background tasks: memory extraction, summarization, cost savings 80-90%.

REQUEST ROUTING BY BACKEND MODULE

Each backend module routes traffic to the optimal model. Percentages reflect expected traffic distribution in a typical production workload. The fallback order (right column) activates via circuit breaker.

Chat / SSE stream	GPT-4o Claude	G → C → G
Tool calls	GPT-4o Claude	G → C
RAG retrieval	GPT-4o Gemini	G → G → C
File ingestion	Gemini GPT-4o	G → G
Memory extraction	Groq	G → G



MODEL → MODULE MAPPING

GPT-4o	Module	Implementation detail
	Chat service	Primary model; best at following complex instructions and multi-turn NLP flows
	Tool execution engine	Best-in-class function calling; routes tool calls through ToolRegistry with JSON Schema
	Workflow LLM steps	Handles conditional logic, structured output extraction, and API orchestration steps
	Embedding (default)	text-embedding-3-small for RAG chunking and semantic memory retrieval
Gemini 1.5 Pro	Module	Implementation detail
	File ingestion — images	Multimodal parsing: PDFs with embedded images, scanned docs, image-based knowledge bases
	Long-context routing	Auto-routes when token count > 70% of GPT-4o window AND content has images/long docs
	Vision tools	browser_automation screenshots, image content in workflow steps, visual document analysis
	Model router signal	Triggered: token_count > 70% context AND content has images OR long documents
Claude 3.5+	Module	Implementation detail
	Narrative & long-form tasks	Document drafting, complex report generation, multi-step reasoning workflows
	Code execution workflows	code_executor tool step — code generation + debugging loops with sandbox isolation
	Human-in-the-loop approvals	Generates approval summaries, synthesizes workflow context for human reviewers
	Fallback provider	Circuit breaker routes here when OpenAI is down; AnthropicProvider normalizes tool_use schema
Groq Llama3	Module	Implementation detail
	Memory extraction (async)	Background Celery task after every response — extract_memories() at 10x lower cost vs GPT-4o
	Conversation summarization	When context hits 70% threshold, Groq summarizes oldest N turns fast and cheap
	Simple query routing	cost_optimization_mode flag routes FAQ-style queries here; 80–90% cost savings per token
	All async Celery workers	Default model for all background AI tasks: memory, summaries, moderation pre-checks

PROVIDER ABSTRACTION LAYER

All providers implement a unified **BaseProvider** abstract class in **packages/ai-sdk/**. The orchestration layer is completely provider-agnostic — it calls the same interface regardless of model.

```
class BaseProvider:
    async def chat_completion(request: ChatRequest) -> ChatResponse
    async def stream_completion(request: ChatRequest) -> AsyncIterator[Token]
    async def embed(texts: List[str], model: str) -> List[List[float]]
    async def tool_call(request: ChatRequest, tools: List[Tool]) -> ToolCallResponse
    def health_check() -> ProviderHealth
    def token_count(messages: List[Message], model: str) -> int
    # Implementations: # OpenAIProvider, AnthropicProvider, GroqProvider, GeminiProvider, VLLMProvider
```

MODEL ROUTER SIGNALS (PRIORITY ORDER)

Signal	Action
User explicitly selected a model	Use that model, ignore all routing
Workspace has a default model configured	Use workspace-configured default
Tool calling required in request	Route to GPT-4o (primary) or Claude (fallback)
token_count > 70% context AND images/long docs	Route to Gemini 1.5 Pro (1M token window)
cost_optimization_mode flag is set	Route simple queries to Groq Llama3
Primary provider circuit breaker open	Automatic failover: GPT-4o → Claude → Groq → 503

CIRCUIT BREAKER & FAILOVER

Each provider has a circuit breaker (half-open / open / closed) implemented with Redis counters. If a provider returns >5 errors in 60 seconds, the circuit opens and traffic reroutes to the next fallback. Circuit retries after 30 seconds.

Primary GPT-4o ● closed	Fallback 1 Claude 3.5 ● standby	Fallback 2 Groq Llama3 ■ half-open	Hard error 503 response ✗ all open
--------------------------------------	--	---	---

Phase 1: Core Chat + Auth + Streaming		Weeks 1–3
BaseProvider abstract class Implement OpenAIProvider + AnthropicProvider with chat_completion(), stream_completion(), token_count(). Normalize message schema.	GPT-4o + Claude	
FastAPI SSE endpoint POST /chat/stream → StreamingResponse. JWT middleware extracts workspace_id. Rate limit via Redis sliding window counters.	Backend	
DB schema: users + conversations Alembic migrations for users, sessions, conversations, messages tables. workspace_id FK enforced on all data models.	PostgreSQL	
Prompt assembly pipeline System prompt → memory context → conv history → user message. Token budget enforced with tiktoken. Context trimmed last.	Orchestration	
Phase 2: Model Routing + Workspaces		Weeks 4–5
Model router logic Route by priority: user selection > workspace default > cost_mode > tool_calling_required > long_context_needed. Redis caches plan limits.	All models	
Groq + Gemini adapters Add GroqProvider and GeminiProvider implementing BaseProvider. Normalize all streaming to AsyncIterator[TokenChunk] interface.	Groq + Gemini	
Context window management Track token count per-request. At 70% → enqueue Celery summarization task using Groq Llama3. Store summary as synthetic message.	Groq (summary)	
Orgs + RBAC memberships organizations, memberships tables. Roles: owner > admin > member > viewer. Enforced at service layer, not just route layer.	Backend	

Phase 3: File Upload + RAG Pipeline		Weeks 6–8
File ingestion Celery worker S3 presigned upload → Celery task → parse (pdfminer/pymupdf, python-docx) → chunk (512 tokens, 50 overlap) → embed → pgvector.		GPT-4o embed
Gemini for image-heavy PDFs If MIME=image/* or PDF has embedded images, route ingestion to Gemini 1.5 Pro. Returns structured text + bounding boxes for chunking.		Gemini
Hybrid retrieval + reranker pgvector cosine search + BM25 full-text search. Merge results. Cohere Rerank or cross-encoder reranker scores and filters top-k chunks.		Retrieval layer
RAG citations in responses Inject top-k chunks with source metadata into prompt. Model returns [source: filename] refs. Frontend renders inline citation badges.		All models
Phase 4: Tool Calling + Integrations		Weeks 9–11
ToolRegistry + execution lifecycle JSON Schema tool definitions. Detect tool_call in SSE stream → pause stream → permission check → sandbox exec → inject result → resume.		GPT-4o primary
Claude as tool-call fallback AnthropicProvider normalizes tool_use schema to unified ToolCall. Same ToolRegistry works identically across both GPT-4o and Claude.		Claude fallback
Code executor sandbox Docker container: no network, read-only FS, CPU/mem/time limits. Claude generates code; sandbox runs it; stdout injected as tool_result.		Claude + sandbox
OAuth integrations (Slack, Gmail) OAuth2 flow → AES-256-GCM encrypted credentials (key in AWS KMS). send_email and slack_message tools registered in ToolRegistry.		Backend
Phase 5: Workflow Automation Engine		Weeks 12–15
DAG runner with Celery workflow_run + workflow_step_run tables. Celery chain/group per step dependencies. Durable: resumes from last completed step on worker restart.		Groq (LLM steps)
LLM step → per-task model routing Each LLM step config specifies preferred_model. Simple steps → Groq, complex/code steps → Claude, vision steps → Gemini.		All models
Human approval step Sets step status = awaiting_approval. Sends signed JWT URL (24h TTL) to approvers. POST /workflow-runs/{id}/approve resumes the DAG.		Claude summary
Memory extraction pipeline After every AI response, Celery enqueues extract_memories(). Groq Llama3 identifies facts → embed → dedup (cosine > 0.95 skip) → insert.		Groq

Phase 6: Billing + Observability + Hardening		Weeks 16–18
Cost tracking per model After each completion: tokens_in, tokens_out, cost_usd recorded in usage_logs. Per-model cost rates in config. Aggregated daily per workspace.	All models	
Langfuse LLM observability Trace every prompt → model call → response. Compare quality/latency/cost across GPT-4o vs Claude vs Groq for identical query types.	Monitoring	
Stripe metered billing Token usage reported hourly via Stripe Usage Records. Plan enforcement: free (50 msg/day), pro (unlimited), team. Redis caches plan status.	Backend	
GDPR + audit logs All auth events, mutations, billing events → audit_logs (partitioned by month). User deletion: cascading soft-delete + async hard-delete PII job.	Backend	

RECOMMENDED TECH STACK

Layer	Technology	Model alignment
Frontend	Next.js 14, TypeScript, Tailwind, shadcn/ui, Zustand, TanStack Query	SSE client for all streaming providers
Backend API	FastAPI, Python 3.12, Pydantic v2, SQLAlchemy 2.0, Alembic	BaseProvider abstract class lives in packages/ai-sdk/
LLM SDKs	openai>=1.50, anthropic>=0.35, groq, google-generativeai	One adapter per provider, same interface
Database	PostgreSQL 16 (RDS), pgvector extension enabled	Vector store for memory + RAG chunks
Queue / Workers	Redis 7, Celery 5.4+ with Redis broker, Celery Beat	Groq runs all async background AI tasks
Object storage	AWS S3 (raw uploads, exports, invoice PDFs)	Gemini reads raw files during ingestion
Observability	Langfuse (LLM traces), Sentry, CloudWatch, OpenTelemetry	Per-model cost, latency p50/p95/p99 tracked
Deployment	Docker, ECS Fargate, GitHub Actions CI/CD, AWS ECR	Model adapters containerized with API service

FIRST 30-DAY ARCHITECTURE-TO-BUILD ROADMAP

Days	Milestone	Model work
1–3	Monorepo setup, Docker Compose (Postgres + Redis), project structure	—
4–7	Auth system: register/login/JWT/refresh; user + workspace DB models; Alembic migrations	—
8–12	Chat foundation: conversation + message models; SSE streaming endpoint	OpenAIProvider + AnthropicProvider adapters
13–16	Frontend: Next.js app shell; auth pages; main chat UI with streaming; conversation sidebar	SSE token accumulator client
17–20	Prompt assembly: context window management; workspace system prompts; conversation summarization	Groq adapter for cheap summarization
21–24	File upload: S3 presigned URLs; file ingestion worker; PDF/DOCX parsing; pgvector chunking	GPT-4o embedding; Gemini for image PDFs
25–27	RAG retrieval: semantic + hybrid search; cross-encoder reranker; citations in responses	All providers for RAG-augmented chat
28–30	Deploy to AWS ECS (staging); CI/CD pipeline; Sentry error tracking; usage logging; internal demo	Model cost tracking in usage_logs

KEY RISKS & MITIGATIONS

Risk	Mitigation
Provider rate limits	Token bucket per provider in Redis; requests queued not dropped; circuit breaker auto-failover
Token counting accuracy	tiktoken for OpenAI/Groq; estimated 1:3.5 char ratio for Anthropic/Gemini; over-budget by 10% for safety
Tool call schema differences	BaseProvider normalizes OpenAI function_call and Anthropic tool_use to unified ToolCall schema
Streaming proxy memory	Tokens never buffered server-side — always proxied directly to client to minimize memory pressure
Embedding cost at scale	Default to text-embedding-3-small; switch to local sentence-transformers for cost optimization phase
Sandbox security	Docker containers: no network, read-only FS, CPU/mem/time limits; webhook URLs validated against allowlist

CAT AI Backend & Model Integration Roadmap v1.0 — Ready for Implementation